ADVANCED DATA STRUCTURES & ALGORITHMS — COURSE PROJECT

Digital Logic Circuit Simulator

Project & Algorithm Mastery Report: Intuitive Explanation, Complete Working & Algorithmic Foundations

Prepared For: Markanda Meher (206126020) & Nikhil (206126021)

Department: Computer Science & Engineering

Review Date: October 12, 2026 (First Review)

Core Focus: Graph Modeling, Cycle Detection, Topological Scheduling, Boolean Simulation

1. The Big Picture in Plain English

This project is a **software simulator for digital logic circuits**. Instead of buying physical breadboards, 7400-series TTL chips, and jumper wires, you can build digital circuits inside software, supply binary inputs (0 or 1), and observe the exact outputs in real-time.

The Real-World Problem We Are Solving

In physical hardware laboratories:

- **Wire Congestion & Loose Pins:** Connecting even 10 logic gates produces an unmanageable tangle of jumper wires. Loose pin contacts cause intermittent faults that take hours to isolate with multimeters.
- **Permanent Hardware Destruction:** Inverted chip insertion, reverse polarity, or short-circuiting an output pin to ground permanently burns 7400-series TTL IC chips.
- **Lack of Design Rule Validation:** Physical breadboards have zero built-in intelligence. If a student wires an output back into its own input (an illegal feedback loop), the circuit oscillates out of control and overheats.
- **Scalability Limitations:** Scaling beyond 15–20 gates on a breadboard becomes physically impractical.

> **The Solution**
>
> We replace physical hardware with **data structures and algorithms**. Logic gates become graph nodes, wires become directed edges, feedback loops are detected via **Depth-First Search**

> **(DFS)**, and gates are scheduled in the mathematically correct order using **Topological Sorting (Kahn's Algorithm)**.

2. System Inputs, Processing, and Outputs

```
+-------------------------------------------------------------------------+
|                                INPUTS                                   |
| 1. Gate Declarations : Gate Type (AND, OR, NOT, XOR...) & ID ("G1")     |
| 2. Wire Connections  : Directed wire from driver gate to load gate      |
| 3. Primary Inputs    : Binary test vectors (A = 1, B = 0)               |
+-------------------------------------------------------------------------+
                                   |
                                   ▼
+-------------------------------------------------------------------------+
|                          PROCESSING PIPELINE                            |
| 1. Graph Construction   : Build Adjacency List & in-degree table       |
| 2. Cycle Detection      : Tarjan's 3-Color DFS catches feedback loops   |
| 3. Evaluation Scheduler : Kahn's In-Degree BFS establishes order        |
| 4. Logic Execution      : Evaluates Boolean formulas in topo order      |
+-------------------------------------------------------------------------+
                                   |
                                   ▼
+-------------------------------------------------------------------------+
|                                OUTPUTS                                  |
| 1. Final Outputs        : (e.g., Sum = 1, Carry = 0)                    |
| 2. Intermediate States  : Logic signal at every internal wire pin       |
| 3. Full Truth Table     : All 2^N combinations evaluated in < 1 ms      |
| 4. Error Diagnostics    : Traps illegal cycles with exact back-edges    |
+-------------------------------------------------------------------------+
```

3. The Core Challenge: Why We Need Smart Algorithms

The "Chicken-and-Egg" Dependency Trap

Suppose you have a simple circuit: Inputs **A** and **B** feed into an **AND gate**, and the AND gate feeds into a **NOT gate**.

If your program stores gates in a simple array: `[NOT_Gate, AND_Gate, Input_A, Input_B]` and runs a basic `for` loop:

1. It tries to calculate **NOT_Gate** first.
2. But NOT_Gate depends on the output of AND_Gate!

3. AND_Gate hasn't been evaluated yet, so it contains uninitialized garbage data.
4. NOT_Gate calculates the completely wrong answer!

> **The Fundamental Rule of Combinational Logic**
>
> A logic gate **CANNOT** evaluate its output until **ALL** of its upstream driver gates have already finished calculating their outputs! To guarantee this order, we must use **Topological Sorting**.

4. Algorithm 1: Graph Representation

We model the circuit as a **Directed Acyclic Graph (DAG)**: $G = (V, E)$.

- **Vertices (V):** Logic gates and primary input switches.
- **Directed Edges (E):** Wires. The directed edge $(u \to v)$ indicates that electrical signal flows from gate u's output into gate v's input.

Data Structures in Memory

```
// 1. Hash Map for instant O(1) lookup of gates by string ID
unordered_map<string, Gate*> gates;

// 2. Adjacency List: "If gate u changes, who receives u's signal?"
unordered_map<string, vector<string>> adjList;

// 3. Predecessor List: "Who feeds into gate v?"
unordered_map<string, vector<string>> predList;

// 4. In-Degree Table: "How many input signals is gate v waiting for?"
unordered_map<string, int> inDegree;
```

Complexity: Building the graph takes $O(V + E)$ time and $O(V + E)$ space.

5. Algorithm 2: Cycle Detection (Tarjan's 3-Color DFS)

Why are cycles forbidden in combinational circuits?

If a NOT gate's output loops back into its input, $Y = \overline{Y}$. If $Y=0 \implies Y=1 \implies Y=0 \dots$ It oscillates infinitely! In hardware, this causes massive current draw. In software, it crashes the program in an infinite recursion loop.

The 3-Color Traffic Light Scheme

Color	State	Meaning
WHITE (0)	Unvisited	Gate hasn't been inspected yet.
GRAY (1)	Active on Call Stack	We entered this gate and are currently traversing its downstream wires.
BLACK (2)	Fully Explored	This gate and all paths downstream of it have been checked and are 100% cycle-free.

> **How a Cycle is Detected**
>
> If the DFS algorithm is exploring from a **GRAY** gate and encounters a neighbor that is already **GRAY**, it means we have looped back onto our active path! This edge is a **Back-Edge**, proving an illegal cycle exists. The simulation immediately halts and reports the offending wire.

```
Algorithm HasCycleDFS(u, colorMap):
    colorMap[u] = GRAY   // Mark active on recursion stack

    For each neighbor v in adjList[u]:
        If colorMap[v] == GRAY:
            ReportError("Cycle found: Back-edge " + u + " → " + v)
            Return TRUE  // Cycle trapped!
        If colorMap[v] == WHITE:
            If HasCycleDFS(v, colorMap) == TRUE:
                Return TRUE

    colorMap[u] = BLACK  // All downstream paths verified safe
    Return FALSE

Time Complexity  : O(V + E)
Space Complexity : O(V) recursion stack depth
```

6. Algorithm 3: Kahn's Topological Sorting (The Scheduler)

Once we confirm the circuit is acyclic, we determine the exact evaluation order using **Kahn's Algorithm (1962)**.

How It Works:

1. **Identify Primary Inputs:** Find all gates with `inDegree == 0` (gates waiting for zero inputs). Push them into a **FIFO Queue**.
2. **Process the Queue:** While the queue is not empty:
   - Pop gate \(u\) from the queue and append it to `evalOrder`.
   - For each downstream gate \(v\) connected to \(u\):
     - Decrement `inDegree[v]` by 1 (representing that one of \(v\)'s inputs is now ready).
     - If `inDegree[v]` drops to 0, all of \(v\)'s prerequisites are met! Push \(v\) into the queue!

```
Algorithm KahnTopologicalSort(gates, inDegree, adjList):
    Q = Queue()
    evalOrder = []

    For each gate g in gates:
        If inDegree[g] == 0:
            Q.push(g)

    While Q is not empty:
        u = Q.pop()
        evalOrder.append(u)

        For each neighbor v in adjList[u]:
            inDegree[v] = inDegree[v] - 1
            If inDegree[v] == 0:
                Q.push(v)

    Return evalOrder

Time Complexity  : O(V + E)
Space Complexity : O(V)
```

7. Algorithm 4: Logic Execution & Truth Table Engine

Topological Simulation Pass

With `evalOrder` ready, we iterate through it from left to right. Every gate calculates its output using the already-settled outputs of its predecessor gates.

Automated Truth Table Generator (\(2^N\) States)

For \(N\) inputs, there are \(2^N\) possible binary combinations. We use bitwise shifting to generate each vector automatically:

```
Algorithm GenerateTruthTable(N_inputs, evalOrder):
    TotalRows = 2^N_inputs
    For row = 0 to (TotalRows - 1):
        For i = 0 to (N_inputs - 1):
            inputVector[Input[i]] = (row >> i) & 1   // Extract i-th bit
        outputVector = SimulatePass(evalOrder, inputVector)
        PrintRow(inputVector, outputVector)
```

8. Complete Step-by-Step Dry Run: The Half Adder



Dry-Run Execution:

1. **Graph:** \(V = \{A, B, G_XOR, G_AND, Sum, Carry\}\). Initial in-degrees: \(A: 0, B: 0, G_XOR: 2, G_AND: 2, Sum: 1, Carry: 1\).
2. **DFS Cycle Check:** Traverses all branches without encountering a GRAY node. Status: ACYCLIC.
3. **Kahn's Sort:**
   - Initial Queue: \([A, B]\).
   - Pop \(A \implies \text{Order} = [A]\). Decrement \(G_XOR\) to 1, \(G_AND\) to 1.
   - Pop \(B \implies \text{Order} = [A, B]\). Decrement \(G_XOR\) to 0 (Enqueued!), \(G_AND\) to 0 (Enqueued!). Queue = \([G_XOR, G_AND]\).
   - Pop \(G_XOR \implies \text{Order} = [A, B, G_XOR]\). Decrement \(Sum\) to 0 (Enqueued!).
   - Pop \(G_AND \implies \text{Order} = [A, B, G_XOR, G_AND]\). Decrement \(Carry\) to 0 (Enqueued!).
   - Pop \(Sum \implies \text{Order} = [A, B, G_XOR, G_AND, Sum]\).
   - Pop \(Carry \implies \text{Order} = [A, B, G_XOR, G_AND, Sum, Carry]\).

4. **Resulting Evaluation Order:** $$\mathbf{[A,\ B,\ G_XOR,\ G_AND,\ Sum,\ Carry]}$$
5. **Simulation for \(A=1, B=1\):**
   - \(A = 1\), \(B = 1\)
   - \(G_XOR = 1 \oplus 1 = \mathbf{0}\)
   - \(G_AND = 1 \cdot 1 = \mathbf{1}\)
   - \(Sum = G_XOR = \mathbf{0}\)
   - \(Carry = G_AND = \mathbf{1}\)
   - Binary sum: \(1 + 1 = 10_2\) (Sum = 0, Carry = 1). 100% mathematically correct!

9. Complexity Summary & Viva Defense Guide

Module / Operation	Algorithm	Time	Space	Core Rationale
Circuit Storage	Adjacency List + Hash Map	\(O(V + E)\)	\(O(V + E)\)	\(O(1)\) lookup by gate name; optimal for sparse circuit graphs.
Cycle Trapping	Tarjan's 3-Color DFS	\(O(V + E)\)	\(O(V)\)	Detects back-edges and isolates feedback loops prior to execution.
Evaluation Scheduling	Kahn's In-Degree BFS	\(O(V + E)\)	\(O(V)\)	Guarantees dependency resolution and prevents race conditions.
Logic Simulation	Topological Boolean Dispatch	\(O(V + E)\)	\(O(V)\)	Evaluates each gate once dependencies are fully settled.
Truth Table Generation	Bitwise Iteration	\(O(2^N \cdot (V + E))\)	\(O(V)\)	Exhaustively verifies all \(2^N\) states in milliseconds.

Top 3 Viva Questions and Simple Answers

Q1: Why Kahn's algorithm instead of DFS for topological sorting?
Answer: Kahn's algorithm works forward from in-degree 0 using a queue (BFS style). This naturally models circuit levelization—gates at the same logic depth can be evaluated in parallel. In-degrees also give an explicit count of remaining inputs.

Q2: Why do you need 3 colors (White, Gray, Black) in DFS instead of just a boolean visited flag?
Answer: A boolean visited flag cannot distinguish between a cross-edge (two independent paths meeting at the same gate, which is completely legal in circuits) and a back-edge (a path looping back to an ancestor, which is an illegal feedback loop). The GRAY state uniquely marks nodes currently on the active recursion stack.

Q3: How did you validate your simulator against industry benchmarks?
Answer: We tested on the official IEEE ISCAS '85 benchmark suite (specifically `c17`, which has 6 NAND gates, 5 inputs, and 2 outputs). We verified all 32 truth table combinations against the IEEE golden standard, matching it with 100% accuracy.